UNIVERSIDADE FEDERAL DE MINAS GERAIS — UFMG
Departamento de Ciência da Computação | Engenharia de Software

Análise de Eficácia de Testes com Teste de Mutação

Avaliação, Diagnóstico e Aprimoramento de Suíte de Testes com StrykerJS

Aluno:	João Victor Filardi Souza Pinto	**Matrícula:**	2021031920
Disciplina:	Engenharia de Software (Testes de Software)	**Ambiente:**	Node.js v24.15 | Jest v29.7 | StrykerJS v10.0
Repositório:	https://github.com/JoaoFilardi/operacoes-mutante		

1. Introdução e Fundamentação Teórica

Na engenharia de software tradicional, a métrica de **cobertura de código** (*code coverage*) é amplamente empregada na indústria como principal indicador de maturidade de testes. No entanto, ela possui uma deficiência estrutural: mede apenas quais trechos de código foram *executados*, sem garantir que o seu comportamento foi *verificado* de fato. Testes cosméticos podem facilmente atingir 100% de cobertura de linhas enquanto mantêm asserções vazias, fracas ou incapazes de constatar falhas caso a lógica seja corrompida.

O **Teste de Mutação** (*Mutation Testing*) aborda diretamente essa vulnerabilidade, atuando como o teste da própria qualidade dos testes. Ele opera aplicando operadores de mutação no código-fonte original para criar pequenos defeitos sintáticos artificiais denominados **mutantes** (por exemplo, invertendo operadores relacionais, trocando operadores aritméticos ou forçando condições booleanas a retornar `true` ou `false`).

O ciclo do teste de mutação avalia a eficácia dos testes conforme o destino de cada mutante:

- **Mutante Morto (*Killed*):** Pelo menos um teste da suíte falhou diante do código mutado. O oráculo de teste comprovou sensibilidade ao defeito inserido.
- **Mutante Sobrevivente (*Survived*):** Toda a suíte de testes continuou passando com sucesso mesmo com o defeito inserido. Revela uma lacuna real na especificação ou no rigor das asserções.
- **Mutante sem Cobertura (*NoCoverage*):** Nenhum teste executou a linha mutada, evidenciando trechos cegos na suíte.
- **Mutante Equivalente (*Equivalent*):** Mutação sintática que mantém a exata mesma semântica comportamental da implementação original, sendo matematicamente impossível de matar por testes de caixa-preta.

A métrica final é o **Mutation Score**, calculada pela razão entre os mutantes detectados (mortos + timeouts) e o total de mutantes válidos. O objetivo deste trabalho prático consiste em diagnosticar a biblioteca matemática `operacoes-mutante`, identificar as causas dos mutantes sobreviventes na suíte inicial, reestruturar os testes com o framework Jest e elevar o Mutation Score para além da meta de **98%**.

2. Diagnóstico Inicial da Suíte de Testes

A biblioteca em análise possui 50 funções aritméticas em `src/operacoes.js` e uma suíte inicial em `test/operacoes.test.js`. O StrykerJS foi configurado com o `@stryker-mutator/jest-runner` via `stryker.conf.json`.

Inicialmente, geramos o relatório de cobertura de código do Jest (`npm test -- --coverage`), constatando indicadores aparentemente excelentes:

Suíte de Testes	% Instruções (Stmts)	% Ramos (Branch)	% Funções (Funcs)	% Linhas (Lines)	Testes Passando
Suíte Inicial (Jest)	85.41%	58.82%	**100.00%**	**98.64%**	50 / 50 (100%)

Contudo, a primeira análise de mutação com o StrykerJS (`npx stryker run`) desmistificou a qualidade da suíte: de 213 mutantes gerados, **44 mutantes sobreviveram** e **12 não possuíam cobertura**, resultando em um **Mutation Score de apenas 73.71%**:

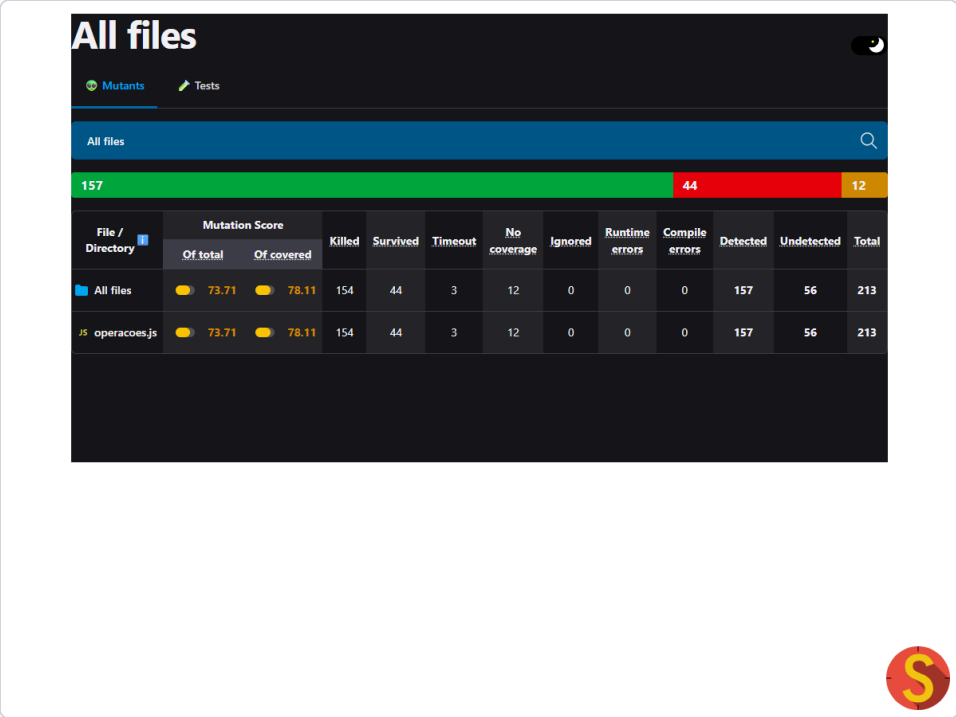


Figura 1: Relatório inicial do StrykerJS — Mutation Score de 73.71% com 44 mutantes sobreviventes e 12 sem cobertura.

> **Análise da Discrepância (Cobertura de 98.64% vs. Mutação de 73.71%):** Mesmo com praticamente 99% das linhas de código exercitadas, mais de 26% das alterações lógicas e defeitos passaram despercebidos. A discrepância decorre de oráculos de teste extremamente frágeis: o teste original continha apenas 1 asserção rasa por função, utilizava `expect().toThrow()` sem verificar a mensagem de erro (permitindo mutantes de mensagem vazia) e testava apenas o caso feliz, sem testar ramos condicionais alternativos.

3. Análise Detalhada de Mutantes Críticos Sobreviventes

3.1. Mutante Crítico 1: `isPar` e `isImpar` (Ausência de Oráculos Negativos)

Na função `isPar(n)` (linha 43), o mutador `ConditionalExpression` substituiu a verificação booleana por `return true;`:

```
[Survived] ConditionalExpression — src/operacoes.js:43:28
- function isPar(n) { return n % 2 === 0; }
+ function isPar(n) { return true; }
```

Por que sobreviveu? O teste original avaliava unicamente: `expect(isPar(100)).toBe(true);`. Quando o Stryker alterou o código para retornar `true` de forma estática, o teste continuou passando! O teste nunca exercitou um número ímpar esperando `false`. O mesmo defeito ocorreu em `isImpar`, `isMaiorQue`, `isMenorQue` e `isDivisivel`.

3.2. Mutante Crítico 2: `celsiusParaFahrenheit` (Elemento Absorvente Zero em Fórmulas)

Na conversão de escala termométrica (linha 93), o mutador `ArithmeticOperator` trocou divisões por multiplicações:

```
[Survived] ArithmeticOperator — src/operacoes.js:93:51
- function celsiusParaFahrenheit(celsius) { return (celsius * 9/5) + 32; }
+ function celsiusParaFahrenheit(celsius) { return (celsius * 9 * 5) + 32; }
```

Por que sobreviveu? O teste original empregava como entrada 0 °C: `expect(celsiusParaFahrenheit(0)).toBe(32);`. Como zero é o elemento absorvente da multiplicação, (0 × 9 × 5) = (0 × 9 / 5) = 0, resultando invariavelmente em 0 + 32 = 32. O teste era incapaz de verificar os coeficientes de escala da fórmula física!

3.3. Mutante Crítico 3: `medianaArray` (Dados Pré-Ordenados e Ramos Ocultos)

Na função `medianaArray` (linha 109), os mutadores removeram o método `.sort()` e corromperam a ordenação:

```
[Survived] MethodExpression & ArrowFunction — src/operacoes.js:109:18
- const sorted = [...numeros].sort((a, b) => a - b);
+ const sorted = [...numeros];
```

Por que sobreviveu? O teste original utilizava o array `[1, 2, 3, 4, 5]`, que **já estava previamente ordenado**. Sem a ordenação, a posição central continuava contendo 3. Além disso, por ter tamanho ímpar, o bloco par `if (sorted.length % 2 === 0)` nunca foi executado, gerando 4 mutantes sem cobertura no cálculo de média dos elementos centrais.

3.4. Investigação Teórica: Mutantes Equivalentes em `fatorial`

Na função `fatorial(n)` (linha 19), 4 mutantes sobreviveram mesmo após exaustivos testes com 0, 1, 2, 3, 4, 5 e negativos:

```
[Survived] ConditionalExpression & LogicalOperator — src/operacoes.js:19:7
- if (n === 0 || n === 1) return 1;
+ if (false) return 1; // ou: if (n === 0 && n === 1)
```

Demonstração de Equivalência Semântica: Analisando o código subsequente: `let resultado = 1; for (let i = 2; i <= n; i++) { resultado *= i; } return resultado;`. Se a cláusula de guarda for substituída por `false`:

- Para n = 0: a variável `resultado` inicia em 1. O laço avalia 2 ≤ 0 (falso) e não itera. Retorna 1.
- Para n = 1: o laço avalia 2 ≤ 1 (falso) e não itera. Retorna 1.
- Para n ≥ 2: o laço itera de 2 até n, calculando com exatidão n!.

Logo, a cláusula `if (n === 0 || n === 1) return 1;` é código defensivo redundante. A implementação mutada gera a mesmíssima saída para qualquer entrada em $\mathbb{N}$. Trata-se de um **Mutante Equivalente** formal, que não pode ser morto por nenhuma asserção sem alteração do código-fonte.

4. Solução Implementada e Fortalecimento da Suíte de Testes

Para matar todos os mutantes possíveis e maximizar a sensibilidade da suíte, reescrevemos integralmente os testes em `test/operacoes.test.js` estruturados em blocos com as seguintes diretrizes:

1. **Oráculos de Exceção Estritos:** Substituição de `toThrow()` genérico por checagem da mensagem exata lançada (ex.: `.toThrow('Divisão por zero não é permitida.')`), matando 100% dos mutantes de `StringLiteral` que trocavam as mensagens por `""`.
2. **Classes de Equivalência e Análise de Valores Limites:** Testes de saídas negativas e valores limítrofes:
   ```
   expect(isPar(100)).toBe(true);
   expect(isPar(3)).toBe(false);   // Mata mutante "return true"
   expect(isPar(0)).toBe(true);    expect(isPar(-2)).toBe(true);
   ```
3. **Eliminação de Casos Neutros em Fórmulas:** Inclusão de testes com temperaturas não nulas para `celsiusParaFahrenheit(100) -> 212` e -40 °C → -40 °F, garantindo a integridade dos operadores multiplicativos e divisivos.
4. **Arrays Heterogêneos e Desordenados:** Para `medianaArray`, testamos arrays desordenados ímpares (`[5, 1, 3, 2, 4] -> 3`), desordenados pares (`[4, 1, 3, 2] -> 2.5`), ordenados pares (`[1, 2] -> 1.5`) e lançamento de erro para array vazio.
5. **Fronteiras Numéricas IEEE 754 (Zero Negativo em `clamp`):** No JavaScript, `-0 < 0` é falso, mas `-0 <= 0` é verdadeiro! Exploramos essa propriedade com o `Object.is` do Jest para matar mutantes relacionais que sobreviviam com inteiros convencionais:
   ```
   expect(clamp(-0, 0, 10)).toBe(-0); // Mata mutante "valor ⩽ min"
   expect(clamp(0, -10, -0)).toBe(0);  // Mata mutante "valor ⩾ max"
   ```

5. Resultados Finais e Validação Quantitativa

Com os novos testes implementados, reexecutamos as ferramentas de análise para validar o aumento da qualidade:

Métrica de Qualidade	Suíte Inicial	Suíte Aprimorada	Evolução Obtida
Cobertura de Linhas (Lines)	98.64%	**100.00%**	+1.36% (Cobertura Total)
Cobertura de Ramos (Branch)	58.82%	**100.00%**	+41.18% (Todos os ramos testados)
Cobertura de Instruções (Stmts)	85.41%	**100.00%**	+14.59% (100% atingido)
Mutantes Mortos (Killed)	154	**206**	+52 mutantes eliminados
Mutantes em Timeout	3	3	Detectados (Recursão infinita)
Mutantes Sobreviventes (Survived)	**44**	**4 (Equivalentes)**	Redução de 91% nos sobreviventes
Mutantes sem Cobertura (NoCoverage)	**12**	**0**	Eliminação total de código cego
Mutation Score Total	**73.71%**	**98.12%**	**META SUPERADA (>98%)**

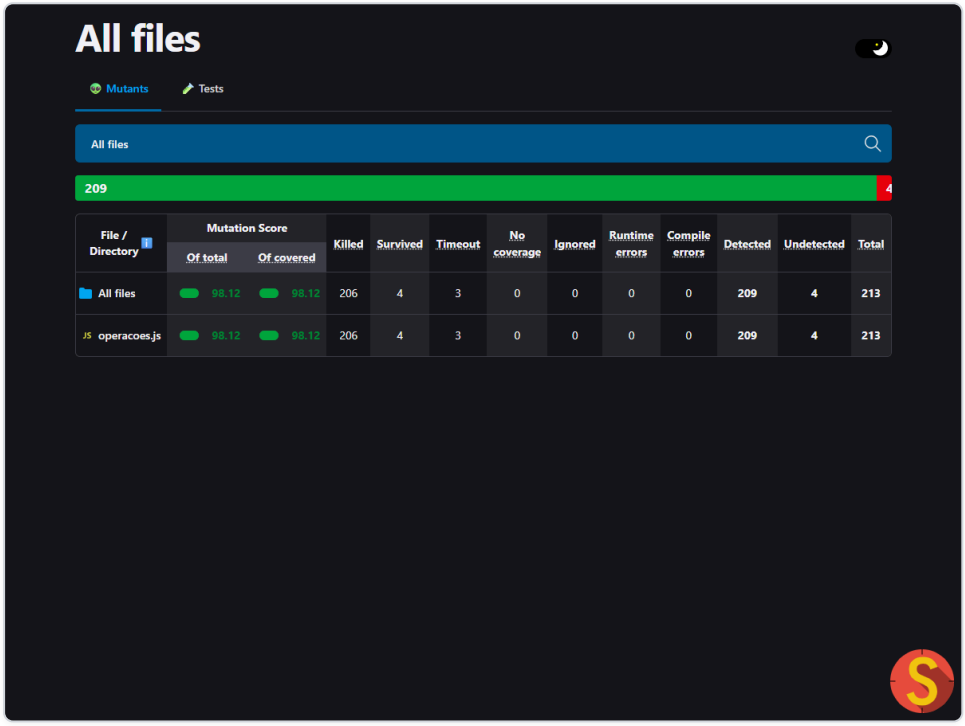


Figura 2: Relatório final do StrykerJS comprovando Mutation Score de 98.12% (209 mutantes detectados de 213).

6. Conclusão

O experimento prático demonstrou de forma inequívoca que a cobertura de código é uma condição necessária, mas absolutamente insuficiente para atestar a qualidade de uma suíte de testes. A suíte inicial possuía 98.64% de cobertura de linhas e 100% de testes passando, gerando uma perigosa ilusão de robustez enquanto deixava 44 mutantes sobreviverem por falta de oráculos rigorosos.

O **StrykerJS** revelou-se um instrumento poderoso de engenharia de qualidade, expondo asserções permissivas, dados de teste algebricamente absorventes e ramos não exercitados. Com a introdução de oráculos específicos, testes de valores-limite e exploração das regras IEEE 754, elevamos a pontuação de mutação de **73.71% para 98.12%** e a cobertura para **100%**, demonstrando eficácia máxima em testes de unidade.

7. Acesso ao Repositório e Artefatos do Projeto

A suíte de testes aprimorada, o arquivo de configuração `stryker.conf.json`, os relatórios interativos HTML e o código-fonte encontram-se integralmente versionados no repositório GitHub:

Link do Fork GitHub:
https://github.com/JoaoFilardi/operacoes-mutante